Mini Coding Agent

핵심 이론과 구현

- 출처: Sebastian Raschka, "*Components of a Coding Agent*"
- 구현: `mini_coding_agent.py` (Python, ~1036 lines)
- 의존성: `openai` 패키지 단 하나

Agent란 무엇인가?

계층 구조

개념	정의
LLM	다음 토큰을 예측하는 기반 모델
Reasoning Model	중간 추론 단계(Chain-of-Thought)에 최적화된 LLM
Agent	모델 + 도구 + 메모리 + 환경 피드백을 사용하는 루프
Agent Harness	context, tool use, prompt, state, 흐름 제어를 관리하는 소프트웨어
Coding Harness	repo 탐색, 코드 편집, 테스트 실행에 특화된 harness

> *"harness가 모델 선택만큼 중요하다"* — Sebastian Raschka

Think-Act-Observe 사이클

```
┌──────────────────────────────────────┐
│              Agent Loop              │
│                                      │
│ [Think]  모델이 전체 프롬프트를 수신  │
│   ↓      다음 행동(도구 호출 or 최종 │
│          답변)을 결정                │
│                                      │
│ [Act]    harness가 도구를 실행       │
│   ↓      (파일 읽기, 코드 작성, 쉘 등)│
│                                      │
│ [Observe] 도구 결과를 히스토리에 추가 │
│   ↓       다시 모델에 전달           │
│                                      │
│ → 모델이 "완료" 응답 시 루프 종료    │
└──────────────────────────────────────┘
```

- 루프 한 번 = 도구 호출 1회 (step)
- `max_steps` 초과 시 강제 종료

전체 아키텍처 — 6개 컴포넌트

#	컴포넌트	역할
1	**WorkspaceContext**	git 메타데이터 수집 → 시스템 프롬프트에 주입
2	**Prompt Shaping**	4-Layer 프롬프트 조립
3	**Tools**	7가지 도구 정의·실행·검증·승인
4	**Context 압축**	토큰 예산 관리 (clip, history 압축)
5	**Session Memory**	JSON 영속화 + 에이전트 루프
6	**Delegation**	읽기 전용 하위 에이전트 위임

Component 1: WorkspaceContext

목적: 에이전트 시작 시 프로젝트 컨텍스트를 수집

```
class WorkspaceContext:
    @staticmethod
    def build(cwd):
        def git(args, fallback=""):
            result = subprocess.run(["git", *args], cwd=cwd,
                capture_output=True, text=True, timeout=5)
            return result.stdout.strip() or fallback

        return WorkspaceContext(
            branch=git(["branch", "--show-current"], "unknown"),
            status=git(["status", "--short"]),
            log=git(["log", "--oneline", "-5"]),
            readme=read_file("README.md")[:1200],  # 최대 1200자
            ...
        )
```

수집 항목

- 현재 브랜치, git status, 최근 5개 커밋
- README.md, AGENTS.md, pyproject.toml (각 최대 1200자)

Component 2: 4-Layer Prompt 구조

```
┌──────────────────────────────────────────────────┐
│ Layer 1: System Prefix (정적, 캐시 재사용)        │
│  - 역할 정의, 규칙, 도구 스키마, 예시             │
│  - WorkspaceContext (git 정보)                   │
├──────────────────────────────────────────────────┤
│ Layer 2: Memory (동적, 매 턴 갱신)                │
│  - task: 현재 작업 (최대 300자)                   │
│  - files: 최근 접근 파일 (최대 8개)               │
│  - notes: 도구 사용 메모 (최대 5개)               │
├──────────────────────────────────────────────────┤
│ Layer 3: Conversation History (동적, 압축)        │
│  - 이전 도구 호출 결과 + 모델 응답                │
│  - 최대 12,000자 (오래된 항목 압축)               │
├──────────────────────────────────────────────────┤
│ Layer 4: Current User Request                    │
│  - 현재 사용자 요청                               │
└──────────────────────────────────────────────────┘
```

- Layer 1은 캐시 재사용 → 비용 절감
- Layer 2~4는 매 API 호출마다 갱신

Component 3: 7가지 Tools

도구	위험성	역할
`list_files`	안전	워크스페이스 파일 목록
`read_file`	안전	파일 내용 읽기 (라인 범위 지정)
`search`	안전	정규식 검색 (ripgrep 또는 Python fallback)
`delegate`	안전	읽기 전용 하위 에이전트 생성
`run_shell`	**위험**	쉘 명령어 실행
`write_file`	**위험**	파일 생성/덮어쓰기
`patch_file`	**위험**	정확한 텍스트 블록 교체

승인 정책 (approval_policy)

- `"ask"` → 사용자에게 `[y/N]` 프롬프트
- `"auto"` → 자동 승인
- `"never"` → 항상 거부 (하위 에이전트 기본값)

Tool 실행 흐름

```
# run_tool(name, args) 흐름
1. 알려진 도구?          → 아니면 error 반환
        ↓
2. validate_tool(name, args)
   - read_file:  파일 존재 + 라인 범위 유효성
   - patch_file: old_text가 정확히 1번 존재
   - run_shell:  timeout 범위 [1, 120]
   - delegate:   depth < max_depth
        ↓
3. repeated_tool_call()?  → 반복이면 error
        ↓
4. 위험 도구?
   approve() → approval_policy 확인
        ↓
5. tool["run"](args) 실행
   → 결과를 최대 4,000자로 clip
```

응답 파싱 — 2가지 형식

JSON 형식 (짧은 인수)

```
<tool>{"name":"read_file","args":{"path":"main.py","end":50}}</tool>
```

XML 형식 (다중 라인 콘텐츠)

```
<tool name="write_file" path="hello.py">
<content>
def greet(name):
    return f"Hello, {name}!"
</content>
</tool>
```

파싱 결과 3가지

```
("tool",  {"name": ..., "args": {...}})  # 도구 호출
("retry", "error message")               # 형식 오류 → 재시도
("final", "answer text")                 # 최종 답변
```

Component 4: Context 압축

문제: 대화가 길어질수록 토큰 초과 위험

```
def history_text(self):
    recent_start = max(0, len(history) - 6)

    for index, item in enumerate(history):
        recent = index >= recent_start
        # 최근 6개: 900자 유지
        # 오래된 항목: 180자로 압축
        limit = 900 if recent else 180

        # 오래된 read_file: 같은 파일 중복 제거
        if not recent and item["name"] == "read_file":
            if path in seen_reads:
                continue
            seen_reads.add(path)

    # 전체 최대 12,000자로 clip
```

요약

- 최근 6개 항목 → 900자 (상세 유지)
- 그 이전 항목 → 180자 (요약)
- 동일 파일 read_file 중복 제거

Component 5: Session JSON 구조

```
{
  "id": "20240406-142033-a1b2c3",
  "workspace_root": "/path/to/repo",
  "history": [
    {"role": "user",      "content": "Create a test file"},
    {"role": "tool",      "name": "list_files", "args": {},
                          "content": "[F] README.md ..."},
    {"role": "assistant", "content": "I've created test.py"}
  ],
  "memory": {
    "task":  "Create a test file",
    "files": ["test_example.py"],
    "notes": ["list_files: [F] README.md [F] ..."]
  }
}
```

- 매 step 후 즉시 디스크에 저장 (`record()` 호출마다)
- `--resume latest` 로 세션 복원 가능

Agent Loop — ask() 핵심 로직

```
def ask(self, user_message):
    record({"role": "user", "content": user_message})

    while tool_steps < max_steps and attempts < max_attempts:
        # 1. 전체 프롬프트를 OpenAI API에 전송
        raw = model_client.complete(self.prompt(user_message))
        kind, payload = self.parse(raw)

        if kind == "tool":       # 도구 호출
            tool_steps += 1
            result = self.run_tool(payload["name"], payload["args"])
            record({"role": "tool", ...result...})
            note_tool(...)
            continue             # 루프 계속

        if kind == "retry":      # 형식 오류
            record(...)
            continue             # 재시도

        # kind == "final": 루프 종료
        record({"role": "assistant", "content": payload})
        return payload
```

메모리 관리 — remember()

3가지 메모리 버킷

버킷	최대 크기	내용
`task`	300자	현재 작업 설명
`files`	8개 항목	최근 접근한 파일 경로
`notes`	5개 항목	도구 실행 요약 메모

```
@staticmethod
def remember(bucket, item, limit):
    if not item:
        return
    if item in bucket:
        bucket.remove(item)   # 중복 제거 후 최신으로 이동
    bucket.append(item)
    del bucket[:-limit]       # 최신 limit개만 유지
```

- FIFO 방식으로 오래된 항목 자동 제거
- 중복 항목 → 제거 후 맨 뒤에 재삽입 (최신화)

Component 6: Delegation

목적: 탐색 작업을 읽기 전용 하위 에이전트에 위임

```
def tool_delegate(self, args):
    if self.depth >= self.max_depth:
        raise ValueError("delegate depth exceeded")

    child = MiniAgent(
        approval_policy="never",          # 위험 도구 자동 거부
        max_steps=int(args.get("max_steps", 3)),  # 최대 3 step
        read_only=True,                   # 파일 쓰기 불가
        depth=self.depth + 1,             # 깊이 추적
    )
    child.session["memory"]["notes"] = [clip(self.history_text(), 300)]
    return "delegate_result:\n" + child.ask(task)
```

재귀 방지 안전장치

장치	메커니즘
읽기 전용	`read_only=True` → `approve()` 항상 False
깊이 제한	`depth >= max_depth` → `delegate` 도구 비활성화
Step 제한	`max_steps=3` 고정

경로 보안 — Path Isolation

목적: 워크스페이스 외부 파일 접근 차단 (`../../../etc/passwd` 등)

```
def path(self, raw_path):
    resolved = (self.root / raw_path).resolve()

    # 경로가 루트 안에 있는지 확인
    if not self.path_is_within_root(resolved):
        raise ValueError(f"path escapes workspace: {raw_path}")
    return resolved

def path_is_within_root(self, resolved):
    probe = resolved
    while not probe.exists() and probe.parent != probe:
        probe = probe.parent
    for candidate in (probe, *probe.parents):
        try:
            if candidate.samefile(self.root):
                return True
        except OSError:
            continue
    return False
```

- 모든 도구 실행 전 경로 검증 필수
- 심볼릭 링크 경로도 `resolve()` 로 정규화

핵심 상수 & 제한값

```
MAX_TOOL_OUTPUT = 4_000    # 도구 출력 최대 길이 (자)
MAX_HISTORY     = 12_000   # 전체 히스토리 최대 길이 (자)

# 히스토리 압축 기준
RECENT_LIMIT    = 900      # 최근 6개 항목: 900자
OLD_LIMIT       = 180      # 오래된 항목: 180자

# 메모리 버킷 크기
TASK_LIMIT      = 300      # task 설명 최대 길이
FILES_LIMIT     = 8        # 최근 파일 목록 최대 개수
NOTES_LIMIT     = 5        # 메모 최대 개수

# 기본 CLI 설정값
--model         gpt-4o-mini
--max-steps     6          # step당 최대 도구 호출 수
--max-new-tokens 512       # 모델 출력 최대 토큰
--temperature   0.2        # 낮은 무작위성
--approval      ask        # 위험 도구 사용자 승인 필요
```

Summary & Key Insight

6개 컴포넌트 요약

- **WorkspaceContext** → 에이전트 시작 시 git/문서 컨텍스트 자동 주입
- **Prompt Shaping** → 4-Layer 구조로 토큰 효율 극대화
- **Tools** → 7가지 도구 + 3단계 안전 검증 (validate → approve → run)
- **Context 압축** → 최근 우선 압축으로 무한 대화 지원
- **Session Memory** → JSON 영속화 + Working Memory 분리
- **Delegation** → 깊이·읽기 전용 제한으로 안전한 병렬 탐색

핵심 인사이트

> *"The harness can often be the distinguishing factor that makes one LLM work better than another."*
> — Sebastian Raschka

- 모델 품질 ≠ 에이전트 품질
- **컨텍스트 품질**이 성능을 결정
- 단일 파일(`mini_coding_agent.py`) 구현 → 학습 최적화